 # Experiment 3:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn import metrics

d=2
N=100
c=2

#import make_classification dataset with two classes and five features from sklearn
xbar, y = make_classification(n_samples=N, n_features=d, n_redundant= 0, n_classes=c)

#Normalize input vectors
xbar = xbar/max(np.linalg.norm(xbar, axis=1))

#Output belongs to +1 or -1 rather than +1 and 0 given by dataset
y=2*y-1

# Split the dataset into 80% training and 20% testing sets
xbar_train, xbar_test, y_train, y_test = train_test_split(xbar, y, test_size=0.2)

# scatter plot for overall dataset

# Scatter plot for class 1
plt.scatter(xbar[y==1, 0], xbar[y==1, 1],
        color='red', marker='o', label='Class 1')

# Scatter plot for class -1
plt.scatter(xbar[y==-1, 0], xbar[y==-1, 1],
```

```
        color='blue', marker='s', label='Class -1')

plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('Overall DataSet Display (2 Features Scatter Plot)')
plt.tight_layout()
plt.show()

# scatter plot for training and testing dataset
# Scatter plot for training data class 1
plt.scatter(xbar_train[y_train==1, 0], xbar_train[y_train==1, 1],
        color='green', marker='o', label='Training Data Class 1')
# Scatter plot for training data class -1
plt.scatter(xbar_train[y_train==-1, 0], xbar_train[y_train==-1, 1],
        color='cyan', marker='s', label='Training Data Class -1')
# Scatter plot for testing data class 1
plt.scatter(xbar_test[y_test==1, 0], xbar_test[y_test==1, 1],
        color='magenta', marker='o', label='Testing Data Class 1')
# Scatter plot for testing data class -1
plt.scatter(xbar_test[y_test==-1, 0], xbar_test[y_test==-1, 1],
        color='black', marker='s', label='Testing Data Class -1')



plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('Training and Testing DataSet Display (2 Features)')
plt.tight_layout()
plt.show()
```

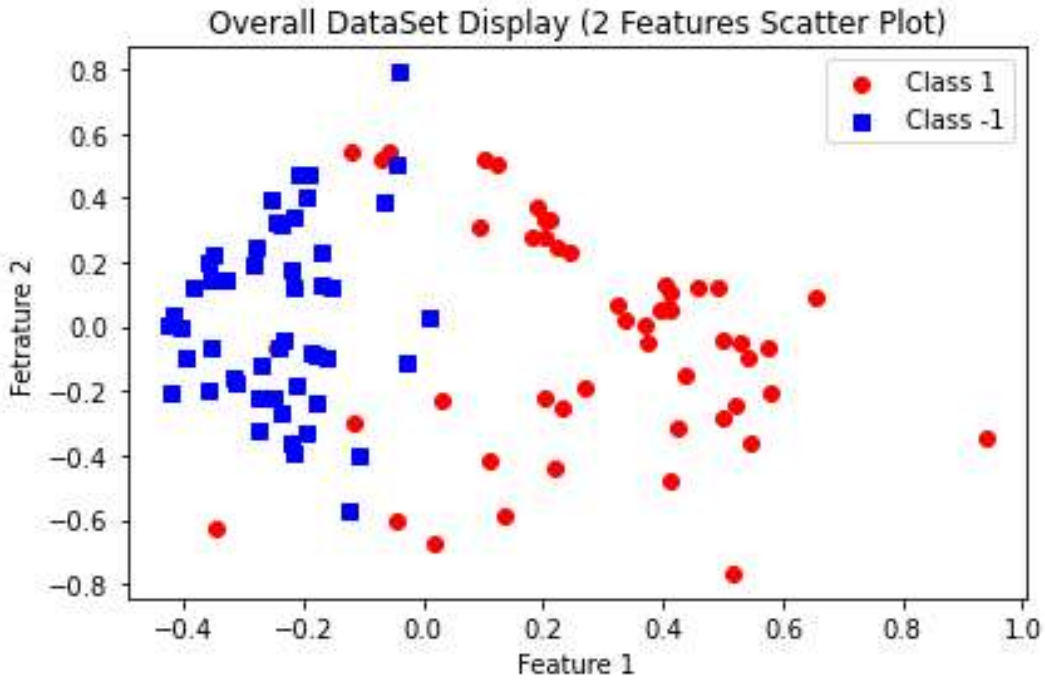
```
# Define sign function
def sign_func(x):
    if(x>0):
        return 1
    else:
        return -1

# Define the perceptron training function with a stopping condition
def perceptron_train(X, y, w,n):

    while True:
        h = sign_func(np.dot(X, w))
        if h!=y:
            w += y*X
            w = w/np.linalg.norm(w)
            n += 1

        else:
            break

    return w,n

# Initialize the weight vector and normalize it
weights = np.random.rand(xbar_train.shape[1])
weights = weights/np.linalg.norm(weights)

# Initialize number of mistakes
n=0
```

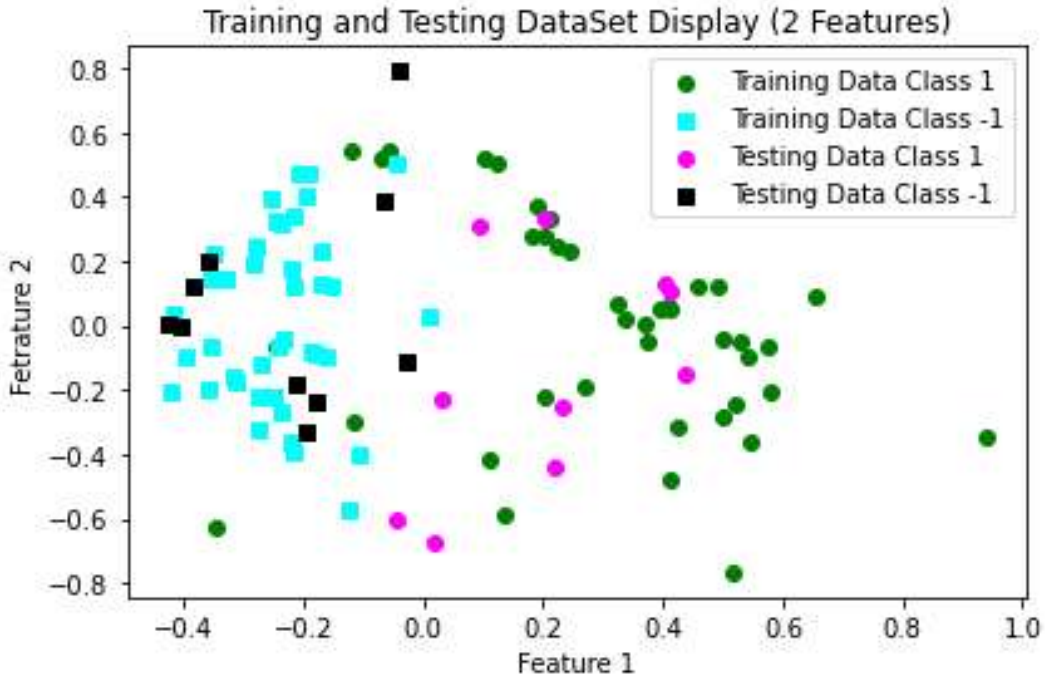
```
# Train the perceptron
for i in range (np.shape(xbar_train)[0]):
    weights,n = perceptron_train(xbar_train[i], y_train[i], weights,n)

print("optimized wieght vector for given training dataset")
print(weights)
print("number of mistakes made in training")
print(n)

# test algorithm of test data set
y_pred=np. empty_like(y_test)
for i in range (np.shape(xbar_test)[0]):
    y_pred[i] = sign_func(np.dot(xbar_test[i], weights))



#plot of testing and predicted dataset
# Scatter plot for testing data class 1
plt.scatter(xbar_test[y_test==1, 0], xbar_test[y_test==1, 1],
            color='magenta', marker='o', label='Testing Data Class 1')
# Scatter plot for testing data class -1
plt.scatter(xbar_test[y_test==-1, 0], xbar_test[y_test==-1, 1],
            color='black', marker='s', label='Testing Data Class -1')
# Scatter plot for predicted data class 1
plt.scatter(xbar_test[y_pred==1, 0], xbar_test[y_pred==1, 1],
            color='brown', marker='o', label='Predicted Data Class 1')
# Scatter plot for predicted data class -1
plt.scatter(xbar_test[y_pred==-1, 0], xbar_test[y_pred==-1, 1],
            color='orange', marker='s', label='Predicted Data Class -1')

# Plot decision boundary with testing and predicted datasets
```

```
x_min, x_max = plt.xlim()

# Generate points along the decision boundary
x_decision_boundary = np.linspace(x_min, x_max, 100)

# Decision boundary equation for 2 features
y_decision_boundary = -(weights[0] * x_decision_boundary)/ weights[1]
plt.plot(x_decision_boundary, y_decision_boundary, color='yellow', label ='decision boundary')
plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Fetrature 2')
plt.title('Testing and Predicted DataSet Display (2 Features) with decision boundry')
plt.tight_layout()
plt.show()

#Finding misclassification error
misclassifications= y_test-y_pred
no_errors = np.count_nonzero(misclassifications)
per_misclassification = no_errors*100/(np.shape(y_test)[0])
print("no. of errors=")
print(no_errors)
print('% misclassification error =')
print (per_misclassification)

# Print accuracy
accuracy = 100-per_misclassification
print(f"Accuracy: {accuracy:.2f}%")

#Find and Plot Confusion Matrix
confusion_matrix = metrics.confusion_matrix(y_test, y_pred)
```

```
cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix, display_labels = [False, True])

cm_display.plot()

plt.show()
```

#Results:



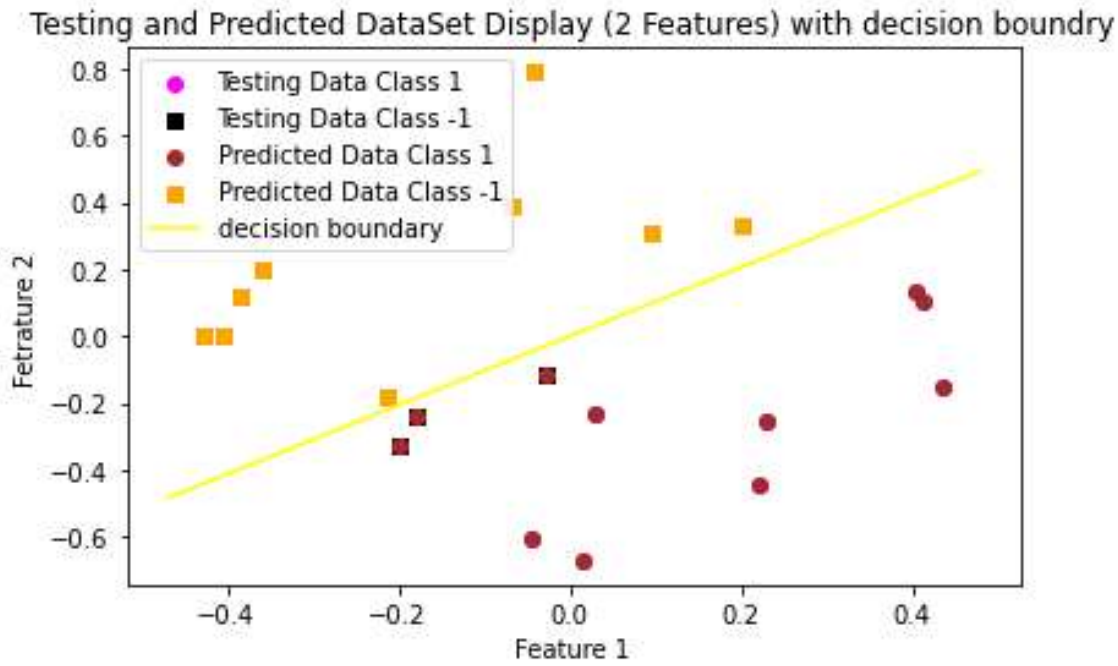


optimized wieght vector for given training dataset

[0.71798295 -0.69606069]

number of mistakes made in training

22



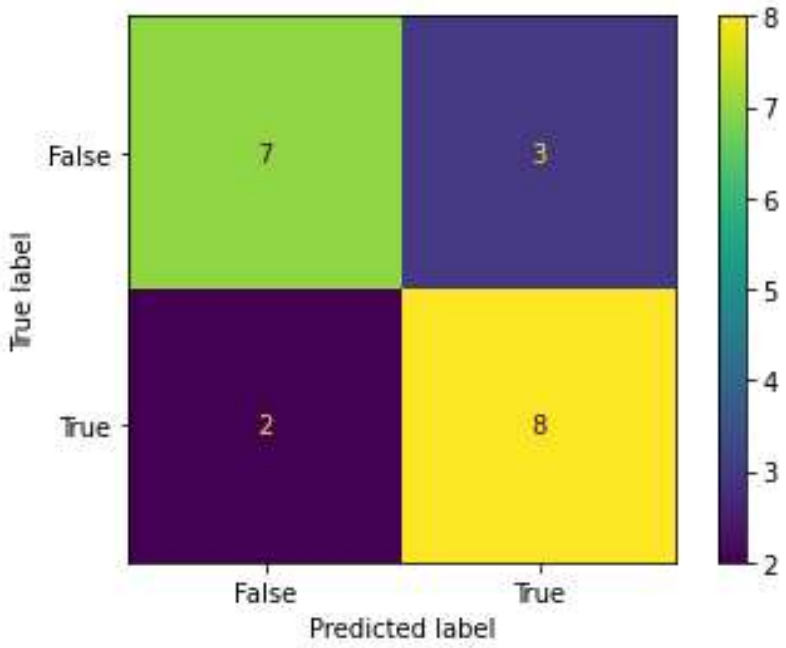
no. of errors=

5

% misclassification error =

25.0

Accuracy: 75.00%


False
True
True label
7
3
2
8
False
True
Predicted label
8
7
6
5
4
3
2